

Cálculo Lambda

Origen

En la década de 1930, varios matemáticos estaban interesados en la pregunta: ¿cuándo una función $f : \mathbb{N} \rightarrow \mathbb{N}$ es **computable**?

Es decir, ¿es posible calcular $f(n)$ para cualquier n dado, usando lápiz y papel?

Se desarrollaron tres modelos de computabilidad por separado:

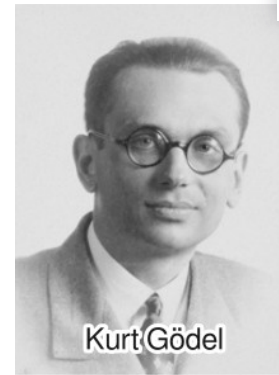
Kurt Gödel: Definió las **Funciones Recursivas Generales** y postuló que una función es computable si y solo si es recursiva general.

Alan Turing: Definió la **Máquina de Turing** y postuló que una función es computable si y solo si es computable por una máquina de Turing.

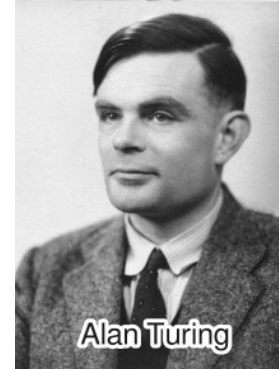
Alonzo Church: Definió el **Cálculo Lambda** y postuló que una función es computable si y solo si puede escribirse como una expresión lambda.

Luego se demostró que estos tres modelos son equivalentes (**Tesis de Church-Turing**).

El Cálculo Lambda jugó un papel importante en el desarrollo de la **teoría de los lenguajes de programación**, particularmente en el **paradigma de programación funcional**.



Kurt Gödel



Alan Turing



Alonzo Church



Funciones

El Cálculo Lambda incorpora dos simplificaciones para operar con funciones:

1. **Las funciones son anónimas.** Por ejemplo, la función

$$\text{suma_cuadrados}(x, y) = x^2 + y^2$$

se puede escribir como

$$(x, y) \mapsto x^2 + y^2$$

2. **Todas las funciones son de un solo argumento.** Por ejemplo, la función

$$(x, y) \mapsto x^2 + y^2$$

se puede escribir como

$$x \mapsto (y \mapsto x^2 + y^2)$$

Esta transformación se llama *currificación* (por Haskell Curry).

Cálculo Lambda

Consiste en escribir **expresiones lambda** con una sintaxis formal, y aplicar **reglas de reducción** para transformarlas.

Una **expresión lambda** puede ser:

- Una **variable**: $x, y, z, \dots$
- Una **abstracción**: $(\lambda x.M)$ donde M es una expresión lambda.
- Una **aplicación**: $(M N)$ donde M y N son expresiones lambda.

Ejemplo: $((\lambda x.(x y)) (\lambda y.(y y))) z$

Una **abstracción** $(\lambda x.M)$ denota una **función anónima** que toma un argumento x y devuelve M , es decir $x \mapsto M$.

Una **aplicación** $(M N)$ denota la acción de **invocar** la función M con el argumento N , es decir $M(N)$.

Convenciones

Para simplificar la notación:

1. Se pueden omitir los paréntesis externos: $(M\ N) \equiv M\ N$
2. Las aplicaciones se asocian a la izquierda: $((M\ N)\ P) \equiv M\ N\ P$
3. El cuerpo de la abstracción se extiende todo lo posible hacia la derecha: $\lambda x.(M\ N) \equiv \lambda x.M\ N$
4. Se pueden contraer múltiples abstracciones lambda: $\lambda x.\lambda y.\lambda z.N \equiv \lambda x\ y\ z.N$
5. Cuando todas las variables son de una única letra, se pueden omitir los espacios: $M\ N\ P \equiv MNP$

Ejemplo:

$(((\lambda x.(\lambda y.(y\ x)))\ a)\ b)$

$((\lambda x.(\lambda y.(y\ x)))\ a)\ b\ (1)$

$(\lambda x.(\lambda y.(y\ x)))\ a\ b\ (2)$

$(\lambda x.\lambda y.y\ x)\ a\ b\ (3)$

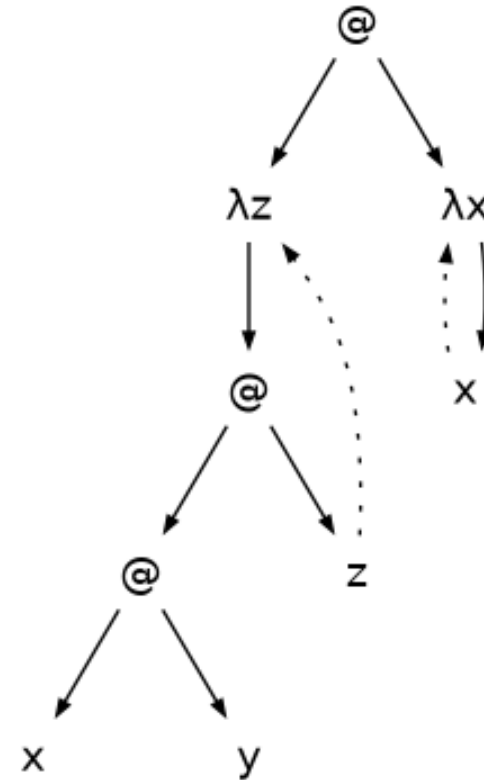
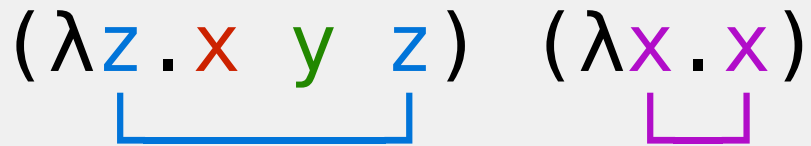
$(\lambda x\ y.y\ x)\ a\ b\ (4)$

$(\lambda xy.yx)ab\ (5)$

Variables libres y ligadas

Una abstracción $\lambda x.M$ **liga** las variables x que aparecen en el cuerpo M . Todas las demás variables en M son **libres**.

$(\lambda z. x \ y \ z) \ (\lambda x. x)$



Reglas de reducción

Conversión Alfa (α): cambiar variables ligadas

En la abstracción $\lambda x.M$, se puede cambiar x por cualquier otra variable que no aparezca libre en M .

Por ejemplo, $\lambda x.x \ y =_{\alpha} \lambda z.z \ y$.

Reducción Beta (β): aplicar una β -redex¹

Una **β -redex** es una aplicación de la forma $(\lambda x.M) \ N$.

Por ejemplo, $(\lambda u.u \ z \ u) \ a =_{\beta} a \ z \ a$.

Reducción Eta (η): Reducir una η -redex

Una **η -redex** es una expresión de la forma $\lambda x.M \ x$ donde x no aparece libre en M .

Según la regla η , $\lambda x.M \ x =_{\eta} M$.

¹“redex” = expresión reducible.

Forma normal

Una expresión lambda está en:

Forma β -normal: si no contiene ninguna β -redex.

Ejemplo: $z (\lambda x. y \ x) \ w$

La forma β -normal de una expresión, si existe, es única.

Forma β - η -normal: si no contiene ninguna β -redex ni η -redex.

Ejemplo: $z (\lambda x. x) \ w$

Forma normal de cabecera: si no hay una β -redex en la cabeza de la expresión.

Ejemplo: $z ((\lambda x. x) \ w)$

Para “evaluar” una expresión lambda, se aplican las reglas de reducción repetidamente hasta llegar a una forma normal.

Estrategias de reducción

Si una expresión contiene más de una redex, ¿cuál se debe reducir primero?

$$\begin{array}{c}
 (\lambda u. u \ (\lambda t. t) \ ((\lambda y. y) \ u)) \ ((\lambda z. z) \ x) \\
 \hline
 \beta \qquad \qquad \beta \\
 \hline
 \beta
 \end{array}$$


Entre las estrategias más comunes están:

Orden normal: siempre se reduce la β -redex más **externa** y a la izquierda.

Si la expresión tiene forma normal, esta estrategia la encuentra.

Orden aplicativo: siempre se reduce la β -redex más **interna** y a la izquierda.

Esta estrategia puede no terminar, incluso si la expresión tiene forma normal.

Call by value: Como el orden aplicativo, pero no se aplican reducciones dentro de abstracciones.

Esta estrategia es la más común en lenguajes de programación tradicionales con **evaluación ansiosa** (C, Python, Java...): los argumentos de una función se evalúan antes de llamar a la función.

Call by name: Como el orden normal, pero no se aplican reducciones dentro de abstracciones.

Esta estrategia y otras similares, son comunes en lenguajes funcionales con **evaluación perezosa** como Haskell: las funciones reciben expresiones sin evaluar, y las evalúan solo cuando las necesitan.

Codificación de tipos de datos

Booleanos

$\text{True} = \lambda x y. x$

$\text{False} = \lambda x y. y$

$\text{Not} = \lambda p. p \text{ False True}$

$\text{And} = \lambda p q. p q \text{ False}$

$\text{Or} = \lambda p q. p \text{ True } q$

$\text{If} = \lambda p q r. p q r$

Numerales de Church

$0 = \lambda f x. x$

$1 = \lambda f x. f x$

$2 = \lambda f x. f (f x)$

$3 = \lambda f x. f (f (f x))$

$\text{Succ} = \lambda n. \lambda f x. f (n f x)$

$\text{Pred} = \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u)$

$\text{Add} = \lambda m n. \lambda f x. m f (n f x)$

$\text{Sub} = \lambda m n. n \text{ Pred } m$

$\text{Mul} = \lambda m n. \lambda f x. m (n f) x$

$\text{Pow} = \lambda m n. \lambda f x. n m f x$

$\text{IsZero} := \lambda n. n (\lambda x. \text{False}) \text{ True}$

$\text{Leq} := \lambda m n. \text{IsZero} (\text{Sub } m n)$

Codificación de listas

Una forma de representar listas enlazadas es construyéndolas con *pares ordenados*:

```
(manzana, (pera, (banana, nil))).
```

Pares:

```
Pair =  $\lambda x y. \lambda s. s \ x \ y$   
Nil := Pair True True
```

```
First =  $\lambda p. p \ \text{True}$   
Second =  $\lambda p. p \ \text{False}$ 
```

Listas:

La lista `[a b c]` se puede representar como
`(False, (a, (False, (b, (False, (c, Nil))))))`

```
Null = IsEmpty = First  
Cons =  $\lambda h. \lambda t. \text{Pair False (Pair h t)}$   
Head =  $\lambda z. \text{First (Second z)}$   
Tail =  $\lambda z. \text{Second (Second z)}$ 
```

Así armamos la lista `[a b c]`:

```
(Cons a (Cons b (Cons c Nil)))
```

Combinadores

Una expresión lambda que no tiene variables libres es un **combinador**.

La **lógica combinatoria** es un modelo de computación equivalente al cálculo lambda pero más simple, ya que usa combinadores en lugar de abstracciones.

Por ejemplo, el **cálculo SKI** se basa en los combinadores:

$$I = \lambda x. x$$

$$K = \lambda x y. x$$

$$S = \lambda x y z. x z (y z)$$

$$I x = x$$

$$K x y = x$$

$$S x y z = x z (y z)$$

Ejemplo: $SKSK = KK(SK) = K$

Recursión

¿Cómo podemos escribir funciones recursivas en el cálculo lambda?

```
Fact = λn.If (IsZero n) 1 (Mul n (Fact (Pred n)))
```

De alguna manera necesitamos que la función pueda invocarse a sí misma. En los lenguajes tradicionales podemos usar el *nombre* de la función dentro del cuerpo de la misma función, pero en el cálculo lambda las funciones no tienen nombre, solo los argumentos.

Recursión (cont.)

Un **combinador de punto fijo** es una función que recibe una función `f` y devuelve un *punto fijo*; es decir un valor `p` tal que `f p = p`.

Es decir, si `Fix` es un combinador de punto fijo:

```
Fix f = f (Fix f)
```

El combinador de punto fijo más conocido es el **combinador Y**:

```
Y = λf.(λx.f (x x)) (λx.f (x x))
```

Usando el combinador Y podemos escribir la función factorial como:

```
Fact = Y (λf.λx.If (IsZero x) 1 (Mul x (f (Pred x))))
```

Recursión (cont.)

El combinador Y produce *stack overflow* en lenguajes con evaluación ansiosa como C y Python.

El combinador Z es una variante que resuelve este problema:

$$Z = \lambda f. (\lambda x. f (\lambda v. x \ x \ v)) (\lambda x. f (\lambda v. x \ x \ v))$$

Implementación del combinador Z en Python:

```
Z = lambda f: (lambda x: f(lambda v: x(x)(v)))(lambda x: f(lambda v: x(x)(v)))

fact = Z(lambda f: lambda n: 1 if n == 0 else n * f(n - 1))

assert fact(5) == 120
```

```
def Z(f):
    def g(x):
        def h(v):
            return x(x)(v)
        return f(h)
    return g(g)

def fact(f):
    def _fact(n):
        return 1 if n == 0 else n * f(n - 1)
    return _fact

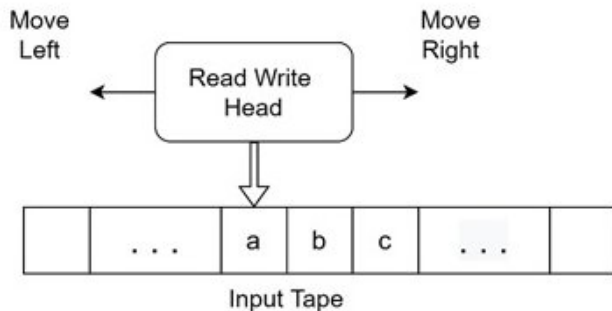
fact = Z(fact)

assert fact(5) == 120
```

La Máquina de Turing [↗](#)

La Máquina de Turing es otro modelo de computación, propuesto por Alan Turing en 1936.

Consiste en una cinta infinita dividida en celdas, un cabezal que puede leer y escribir símbolos en la cinta, y una máquina de estados interna que controla el movimiento del cabezal y las operaciones de lectura/escritura.



La Máquina de Turing y Cálculo Lambda son modelos equivalentes, pero opuestos en su enfoque:

- Máquina de Turing → Lenguajes imperativos
- Cálculo Lambda → Lenguajes funcionales

Tesis de Church-Turing [↗](#)

Una función es computable si y solo si es computable por una máquina de Turing.

Se puede probar que el cálculo lambda y la máquina de Turing son modelos equivalentes, Es decir, que cualquier función que se puede computar con una máquina de Turing también se puede computar con el cálculo lambda, y viceversa.

Generalizando, se dice que un modelo es **Turing-completo** [↗](#) si puede simular a una máquina de Turing, lo que implica que es capaz de procesar cualquier algoritmo computable.

Ejemplos: [↗](#)

- La máquina de Turing
- Cálculo Lambda
- Lógica combinatoria
- La arquitectura de von Neumann [↗](#)
- La mayoría de los lenguajes de programación (ej: Python, C, Prolog, Haskell)
- Autómatas celulares (ej: Game of Life [↗](#))
- Minecraft [↗](#)

www.ingenieria.uba.ar

f    /ingenieriauba

 /FIUBAoficial